Software Requirements and Design Document

For

Group Hoos' Fishin'

Version 1.0

Authors:
Juan Pablo Pastrana II
Tyler Dussault
Bridget Gizzi
Darius Davis
Dhruv Patel

1. Overview (5 points)

Hoo's Fishin' is a 2D fishing game where the player controls an owl named Hoo. The main gameplay loop centers on finding and catching fish, storing catches, selling fish to earn money, and improving fishing equipment. Different fish have their own values, rarity, and catch difficulty, while fishing rods affect which fish can be caught and how the fishing minigame works. As the player progresses, they can purchase upgrades, manage additional inventory space, and complete fishing tasks to earn rewards.

The game includes systems for player movement, fish spawning, fishing minigames, inventory and hotbar management, a backpack, shop interactions, money, and equipment upgrades. It also tracks the player's fishing progress and provides tasks that encourage catching different fish. These systems work together to create a gameplay experience focused on fishing, earning money, and gradually improving the player's equipment and abilities.

2. Functional Requirements (10 points)

The following functional requirements describe the functionality implemented in Hoo's Fishin' during Increments 1 and 2. The requirements are separated by increment to show how the game's functionality has expanded throughout development. Each requirement is assigned a high, medium, or low priority.

Increment 1

The following functional requirements describe the features currently implemented or supported in the increment 1 version of Hoo's Fishin' based on the current project code. These requirements reflect the functionality available at this stage of development and may be expanded ore changed in later increments.

1. High - The system lets the player move Hoo around the game world
2. High - The system lets the player interact with fish in the game world.
3. High - The system starts the fishing minigame when the player selects a fish.
4. High - The system determines whether a fishing attempt is successful or unsuccessful.
5. High - The system adds a successfully caught fish to the player's inventory.
6. High - The system stores multiple copies of the same fish in one inventory slot when stacking is available.
7. High - The system displays the player's inventory items in the hotbar.
8. Medium - The system displays the quantity of stacked items in the hotbar.
9. Medium - The system lets the player switch between hotbar slots.
10. Medium - The system displays the name of the currently selected item.
11. High - The system lets the player sell caught fish.
12. High - The system adds the value of a sold fish to the player's total money.
13. Medium - The system displays the player's current amount of money.
14. Medium - The system supports different fish with different names, values, and fishing difficulty.
15. Medium - The system spawns fish into the game world.
16. Medium - The system allows fish to move within the game world.
17. Medium - The system supports fishing rods with different attributes.
18. Medium - The system shall make the quipped fishing rod affect the fishing minigame..
19. Medium - The system supports purchasing or upgrading fishing equipment.
20. High - The system prevents purchases when the player does not have enough money.
21. Low - The system lets the player pause and resume the game.
22. Low - The system shall let the player quit the game through the pause menu.

Increment 2

23. Medium - The system shall display a main menu that allows the player to start or quit the game.
24. Low - The system shall provide a day/night cycle that changes the appearance of the game world over time.
25. High - The system shall allow the player to purchase a backpack to increase inventory storage.
26. High - The system shall allow the player to move items between the backpack and hotbar.
27. Medium - The system shall allow the player to purchase backpack upgrades to increase storage capacity.
28. Medium - The system shall prevent the backpack from opening while the player is fishing or interacting with the shop.
29. Medium - The system shall track the total number of fish caught by the player.
30. Medium - The system shall track the number of fish caught for each species.
31. Medium - The system shall record whether a fish species has been discovered by the player.
32. High - The system shall provide fishing tasks that the player can accept and complete.
33. High - The system shall update task progress when the player catches fish that meet the task requirements.
34. Medium - The system shall reward the player with money when a task is completed.
35. Medium - The system shall allow the player to switch between the shop and tasks interfaces.
36. Medium - The system shall adjust the fishing minigame's catch zone based on the equipped rod's level.
37. Medium - The system shall support different fishing minigames for certain fish.
38. High - The system shall prevent the player from moving while fishing.
39. Medium - The system shall prevent the player from interacting with the shop while fishing.

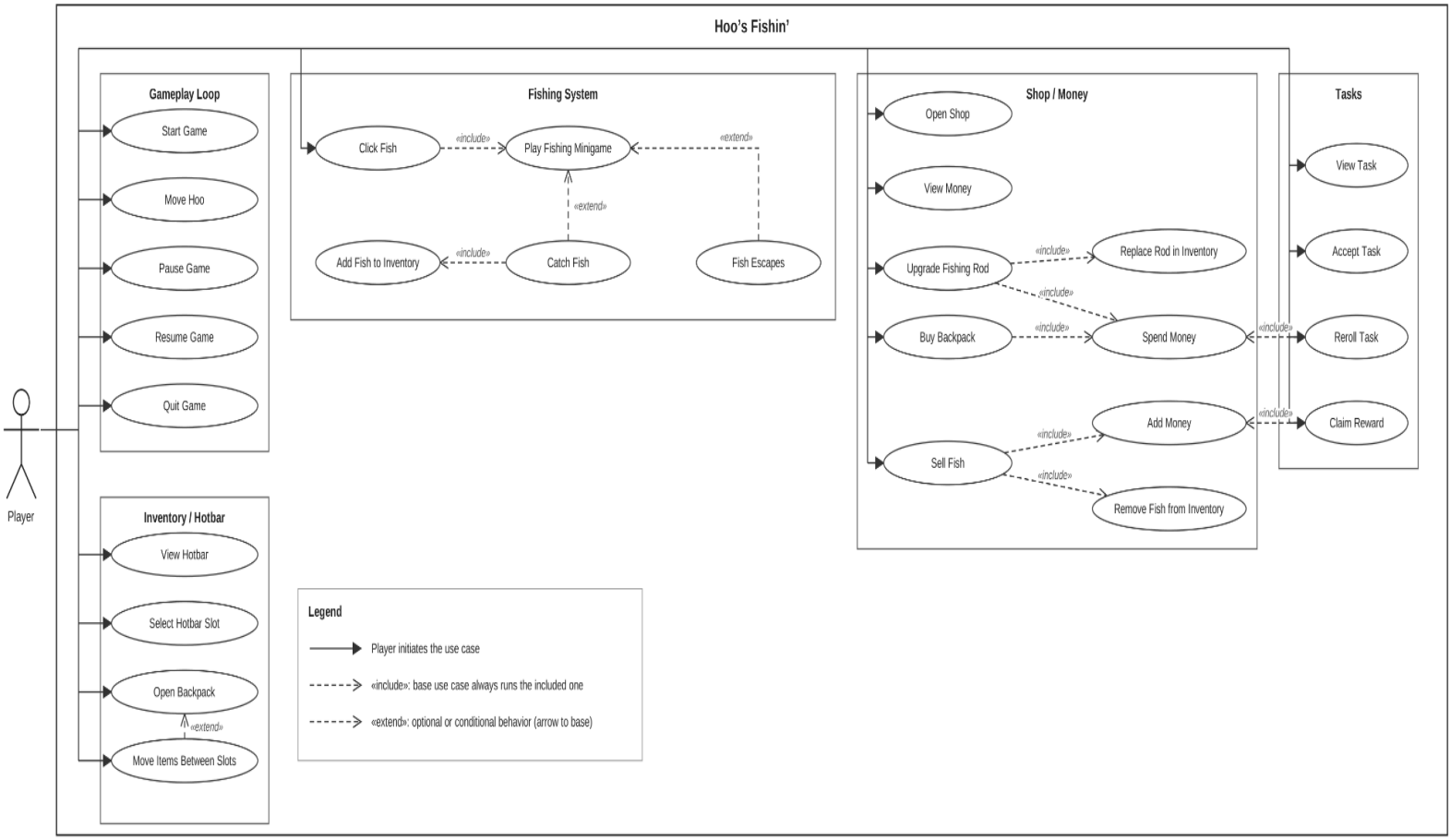
3. Non-functional Requirements (10 points)

1. Look and feel
    a. The system is visually consistent such as being 2D pixel art that has no blur and wrong scaling.
    b. The system shall maintain a consistent user interface across the inventory, hotbar, backpack, shop, and tasks.
    c. The system shall display game elements clearly without overlapping or obstructing important information.
2. Maintainability
    a. The system shall store game data using Godot resources so that new fish and fishing rods can be added without major changes to existing code.
    b. he system shall organize gameplay systems so that new features and upgrades can be added without modifying multiple unrelated components.
3. Reliability
    a. The system does not crash.
    b. The system does not allow users to achieve negative balances.
    c. The system does not allow inventory counts to be inaccurate.
    d. The system shall maintain accurate player progress when tracking fish catches and completing tasks.
4. Usability
    a. The system allows new users to pick up the game and learn how to play without having to be told externally.
    b. The system allows controls to be consistent and information is displayed such as balance and items.
    c. The system shall make it easy for players to navigate between the shop and tasks and manage items in the backpack and hotbar.
5. Performance

a. The system allows for the game to run at 60 frames per second.
b. The system shall handle fish spawning, movement, and other gameplay systems without noticeable delays or significant performance issues.

4. Use Case Diagram (10 points)

Use Case Diagram



Textual descriptions of use cases

Use Case: Start Game

Actors: Player, System

Pre condition: The game is launched and the main menu is shown.

Normal Flow: The player presses Play on the main menu. The system loads the main game scene and spawns Hoo, the fish, the shopkeeper and the HUD.

Post condition: The world is running and the player controls Hoo.

Alternative Flows: The player presses Quit instead and the game closes (see Quit Game).

Nonfunctional Requirements: The system does not crash and loads the game world without noticeable delay.

Use Case: Move Hoo

Actors: Player, System

Pre condition: The game is running and not paused.

Normal Flow: The player holds A / D or the Left / Right arrow. The system moves Hoo at his speed and updates his animation. When the player releases the key, Hoo stops.

Post condition: Hoo's position and animation are updated.

Alternative Flows: While the player is fishing, the system prevents Hoo from moving. While the game is paused, movement input is ignored.

Nonfunctional Requirements: The game runs at 60 frames per second and the controls respond consistently.

Use Case: Pause Game

Actors: Player, System

Pre condition: The game is running and not paused.

Normal Flow: The player presses Esc. The system pauses the game and shows the pause menu with Resume and Quit buttons.

Post condition: The game is paused and the pause menu is visible.

Alternative Flows: The player presses Esc again to resume (see Resume Game).

Nonfunctional Requirements: The pause menu is visually consistent with the rest of the user interface.

Use Case: Resume Game

Actors: Player, System

Pre condition: The game is paused.

Normal Flow: The player presses Resume or Esc. The system unpauses the game and hides the pause menu.

Post condition: The game is running again with all player progress unchanged.

Alternative Flows: N/A

Nonfunctional Requirements: The system is reliable and keeps accurate player progress across a pause.

Use Case: Quit Game

Actors: Player, System

Pre condition: The main menu or the pause menu is open.

Normal Flow: The player presses Quit. The system closes the game.

Post condition: The application is closed.

Alternative Flows: The player chooses Resume instead and keeps playing.

Nonfunctional Requirements: The system closes cleanly without crashing.

Use Case: Click Fish

Actors: Player, System, Fish spawn manager, Swimming fish

Pre condition: The game is running, a fish is swimming, and no fishing minigame is active.

Normal Flow: The player left-clicks a swimming fish. The fish tells the Fish spawn manager it was clicked. The manager checks that the equipped rod can catch that fish, hooks it so it freezes in place, and starts the fishing minigame (see Play Fishing Minigame).

Post condition: The fish is hooked, the minigame is running, and the player is marked as fishing.

Alternative Flows: A minigame is already running, or the equipped rod cannot catch that fish, so the click is ignored and the fish is not hooked.

Nonfunctional Requirements: The system is reliable and only one fish can be hooked at a time. Fish spawning and movement run without noticeable delays.

Use Case: Play Fishing Minigame

Actors: Player, System, Fishing minigame, Fishing rod

Pre condition: A fish is hooked.

Normal Flow: The system starts the minigame using the fish's catch difficulty, with the equipped rod's speed multiplier and catch zone (based on rod level) applied. The player holds the reel input (left mouse button, Space or Enter). The system updates the progress and then decides whether the attempt succeeded or failed.

Post condition: The result of the attempt is known and the player is no longer marked as fishing.

Alternative Flows: Some fish use a different minigame, such as the Flappy minigame, which is set by the fish's data. A successful attempt leads to Catch Fish and a failed attempt leads to Fish Escapes.

Nonfunctional Requirements: The minigame runs at 60 frames per second, and the equipped rod noticeably changes how hard it is.

Use Case: Catch Fish

Actors: Player, System, Fish spawn manager, Player inventory, Catch tracker, Task board

Pre condition: The fishing minigame ended in success.

Normal Flow: The system adds the fish to the inventory (see Add Fish to Inventory). The Catch tracker records the total and per-species counts and marks the species as discovered. The Task board adds to its progress if the fish meets the task requirements. The fish is removed from the water.

Post condition: The fish is in the inventory and the catch counts and task progress are updated.

Alternative Flows: The inventory is full, so the catch fails and the fish escapes (see Fish Escapes).

Nonfunctional Requirements: The system keeps inventory counts, catch counts and task progress accurate.

Use Case: Fish Escapes

Actors: Player, System, Fish spawn manager, Swimming fish, Task board

Pre condition: The fishing minigame ended in failure, or the inventory was full after a success.

Normal Flow: The Fish spawn manager tells the Task board about the failed catch, which resets any catch streak. The manager makes the fish flee. The fish swims away and despawns.

Post condition: The player gains no item and the fish is gone from the water.

Alternative Flows: N/A

Nonfunctional Requirements: The system is reliable and light on resources, and removes escaped fish so they do not build up in the world.

Use Case: Add Fish to Inventory

Actors: System, Player inventory

Pre condition: A fish was caught.

Normal Flow: The system adds the fish to the inventory. The inventory puts it in a matching stack (up to 64) or an empty slot and returns the number added. The hotbar and backpack then refresh.

Post condition: The fish count in the inventory has increased.

Alternative Flows: There is no room for the fish, so the inventory returns 0 and the catch fails.

Nonfunctional Requirements: The system does not allow inventory counts to be inaccurate.

Use Case: View Hotbar

Actors: Player, System, Hotbar

Pre condition: The game is running.

Normal Flow: The hotbar shows its three slots with item icons and the quantity of stacked items. It highlights the selected slot, shows the name of the selected item, and refreshes whenever the inventory changes.

Post condition: The hotbar matches the contents of the inventory.

Alternative Flows: N/A

Nonfunctional Requirements: The hotbar is consistent with the rest of the interface and does not hide important information.

Use Case: Select Hotbar Slot

Actors: Player, System, Hotbar

Pre condition: The game is running.

Normal Flow: The player presses 1, 2 or 3, or clicks a slot. The hotbar highlights the chosen slot and sets the player's current item to what is in it.

Post condition: The chosen slot is active and the name of its item is displayed.

Alternative Flows: The slot is empty, so the player holds nothing.

Nonfunctional Requirements: The controls are consistent and respond immediately.

Use Case: Open Backpack

Actors: Player, System, Backpack UI, Player inventory

Pre condition: The player has bought the backpack.

Normal Flow: The player presses B. The system shows the backpack panel with its slots. The player presses B again to close it.

Post condition: The backpack panel is shown or hidden.

Alternative Flows: The player does not own a backpack, so nothing opens. The system also does not open the backpack while the player is fishing or using the shop.

Nonfunctional Requirements: The backpack is easy to use and matches the style of the hotbar and shop.

Use Case: Move Items Between Slots

Actors: Player, System, Backpack UI, Player inventory

Pre condition: The backpack is open.

Normal Flow: The player clicks one slot and then a second slot. The inventory swaps their contents, and the hotbar and backpack refresh.

Post condition: The contents of the two slots are swapped.

Alternative Flows: The player clicks the same slot twice and nothing changes.

Nonfunctional Requirements: No items are lost or duplicated, and inventory counts stay accurate.

Use Case: Open Shop

Actors: Player, System, Shop keeper

Pre condition: Hoo is within range of the shopkeeper and the player is not fishing.

Normal Flow: The player walks up to the shopkeeper and presses the shop button. The system marks the player as shopping and shows the shop menu with the Shop and Tasks tabs.

Post condition: The shop menu is open.

Alternative Flows: The player is fishing, so the system does not let them use the shop. The player walks out of range and the menu closes.

Nonfunctional Requirements: The player can easily switch between the shop and tasks tabs.

Use Case: View Money

Actors: Player, System, Player wallet

Pre condition: The shop menu is open.

Normal Flow: The system reads the wallet balance and displays it. It updates the display after every transaction.

Post condition: The balance shown is the current balance.

Alternative Flows: N/A

Nonfunctional Requirements: The balance is always clearly displayed and never negative.

Use Case: Sell Fish

Actors: Player, System, Shop keeper, Player inventory, Player wallet

Pre condition: The shop menu is open and the player has a fish selected in the hotbar.

Normal Flow: The player presses Sell. The Shop keeper reads the selected slot and checks it holds a fish. It adds the fish's sell value to the wallet (see Add Money), removes one fish from the slot (see Remove Fish from Inventory), and shows "Sold for $X".

Post condition: The player's money has increased and the fish stack is one smaller.

Alternative Flows: The slot is empty, so the system shows "No fish". The item is not a fish, so the system shows "Select Fish".

Nonfunctional Requirements: The system is reliable and keeps the balance and inventory counts accurate.

Use Case: Upgrade Fishing Rod

Actors: Player, System, Shop keeper, Fishing rod, Player wallet, Player inventory

Pre condition: The shop menu is open and the equipped rod has a next upgrade.

Normal Flow: The player presses Buy. The Shop keeper loads the next rod (Old Reliable, then Papa New Toy for $250, then The Claw 3000 for $1,200) and checks the player can afford it. It takes the money (see Spend Money), replaces the old rod in the inventory (see Replace Rod in Inventory), equips the new rod and shows "Bought" with the rod's name.

Post condition: The new rod is equipped and the player's money is reduced by its price.

Alternative Flows: The rod is already the best one, so the system shows "Max Rod". The player cannot afford it, so the system shows "Need $X" and the rod is not upgraded.

Nonfunctional Requirements: The system does not allow a negative balance, and new rods can be added through Godot resources without major code changes.

Use Case: Buy Backpack

Actors: Player, System, Shop keeper, Player wallet, Player inventory

Pre condition: The shop menu is open, the player does not own a backpack, and they have at least $200.

Normal Flow: The player presses Buy Backpack. The system takes the $200 (see Spend Money) and the inventory unlocks the backpack slots. The button changes to “Backpack owned” and the system shows “Backpack equipped! Press B to open.”

Post condition: The player owns the backpack and can open it (see Open Backpack).

Alternative Flows: The player already owns the backpack, so nothing is bought. The player cannot afford it, so the purchase fails and no money is spent.

Nonfunctional Requirements: The system does not allow a negative balance and keeps inventory counts accurate.

Use Case: Add Money

Actors: System, Player wallet

Pre condition: A fish sale or a task reward is being paid out.

Normal Flow: The system adds the amount to the wallet and the balance increases.

Post condition: The wallet balance has increased by that amount.

Alternative Flows: N/A

Nonfunctional Requirements: The system is reliable and keeps the balance accurate.

Use Case: Spend Money

Actors: System, Player wallet

Pre condition: A rod upgrade, backpack purchase or task reroll is being paid for.

Normal Flow: The system checks that the balance covers the price and then takes the price out of the wallet.

Post condition: The wallet balance is reduced by the price.

Alternative Flows: The balance is too low, so nothing is taken and the purchase stops.

Nonfunctional Requirements: The system does not allow the balance to go negative.

Use Case: Remove Fish from Inventory

Actors: System, Player inventory

Pre condition: A fish in a slot is being sold.

Normal Flow: The system removes one fish from the slot. The inventory lowers the stack count, or clears the slot if it was the last fish. The hotbar and backpack refresh.

Post condition: The slot holds one fewer fish.

Alternative Flows: N/A

Nonfunctional Requirements: The system does not allow inventory counts to be inaccurate.

Use Case: Replace Rod in Inventory

Actors: System, Player inventory, Fishing rod

Pre condition: A rod purchase has been paid for.

Normal Flow: The system removes the old rod from its slot and puts the new rod in its place. The hotbar then refreshes.

Post condition: The inventory holds the new rod instead of the old one.

Alternative Flows: N/A

Nonfunctional Requirements: No items are lost or duplicated during the swap.

Use Case: View Task

Actors: Player, System, Shop keeper, Task board

Pre condition: The shop menu is open.

Normal Flow: The player opens the Tasks tab. The system shows the task's objective, the progress toward its goal and its reward. It also shows a countdown for timed tasks and labels the action button "Accept task", "In progress" or "Claim reward".

Post condition: The current task details are displayed.

Alternative Flows: N/A

Nonfunctional Requirements: The player can easily move between the shop and tasks tabs, and the interface stays consistent.

Use Case: Accept Task

Actors: Player, System, Shop keeper, Task board

Pre condition: A task is being offered.

Normal Flow: The player presses Accept task. The Task board sets the task to active with no progress and starts a countdown if the task is timed. The button changes to “In progress”. Each catch that meets the task requirements adds to the progress until the goal is reached and the task is ready to claim.

Post condition: The task is active, or ready to claim once the goal is met.

Alternative Flows: A timed task runs out of time and goes back to being offered with a “Time up” message. A failed catch resets the progress of a streak task.

Nonfunctional Requirements: The system keeps task progress accurate and does not let it be lost between catches.

Use Case: Reroll Task

Actors: Player, System, Shop keeper, Task board, Player wallet

Pre condition: A task is being offered and the player has at least $25.

Normal Flow: The player presses New offer ($25). The system takes the $25 (see Spend Money). The Task board rolls a different task and the page refreshes.

Post condition: A new task is offered and $25 has been spent.

Alternative Flows: The player has less than $25, so the system shows “Need $25 to reroll” and keeps the current offer.

Nonfunctional Requirements: The system does not allow a negative balance.

Use Case: Claim Reward

Actors: Player, System, Shop keeper, Task board, Player wallet

Pre condition: The task has reached its goal and is ready to claim.

Normal Flow: The player presses Claim reward. The Task board works out the payout and the system adds it to the wallet (see Add Money). The Task board then offers the next task and the system shows “Received $X!”.

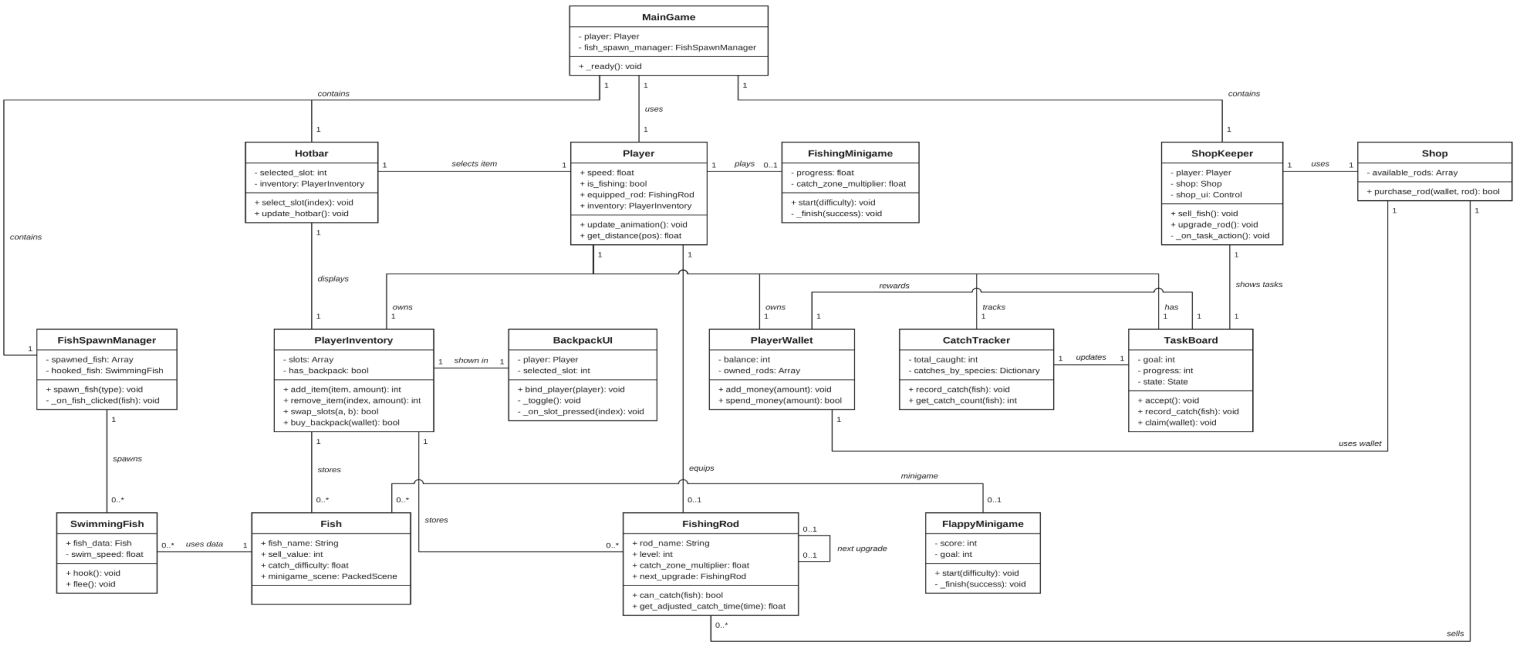
Post condition: The player’s money has increased and a new task is offered.

Alternative Flows: The task is not ready, so the player cannot claim a reward.

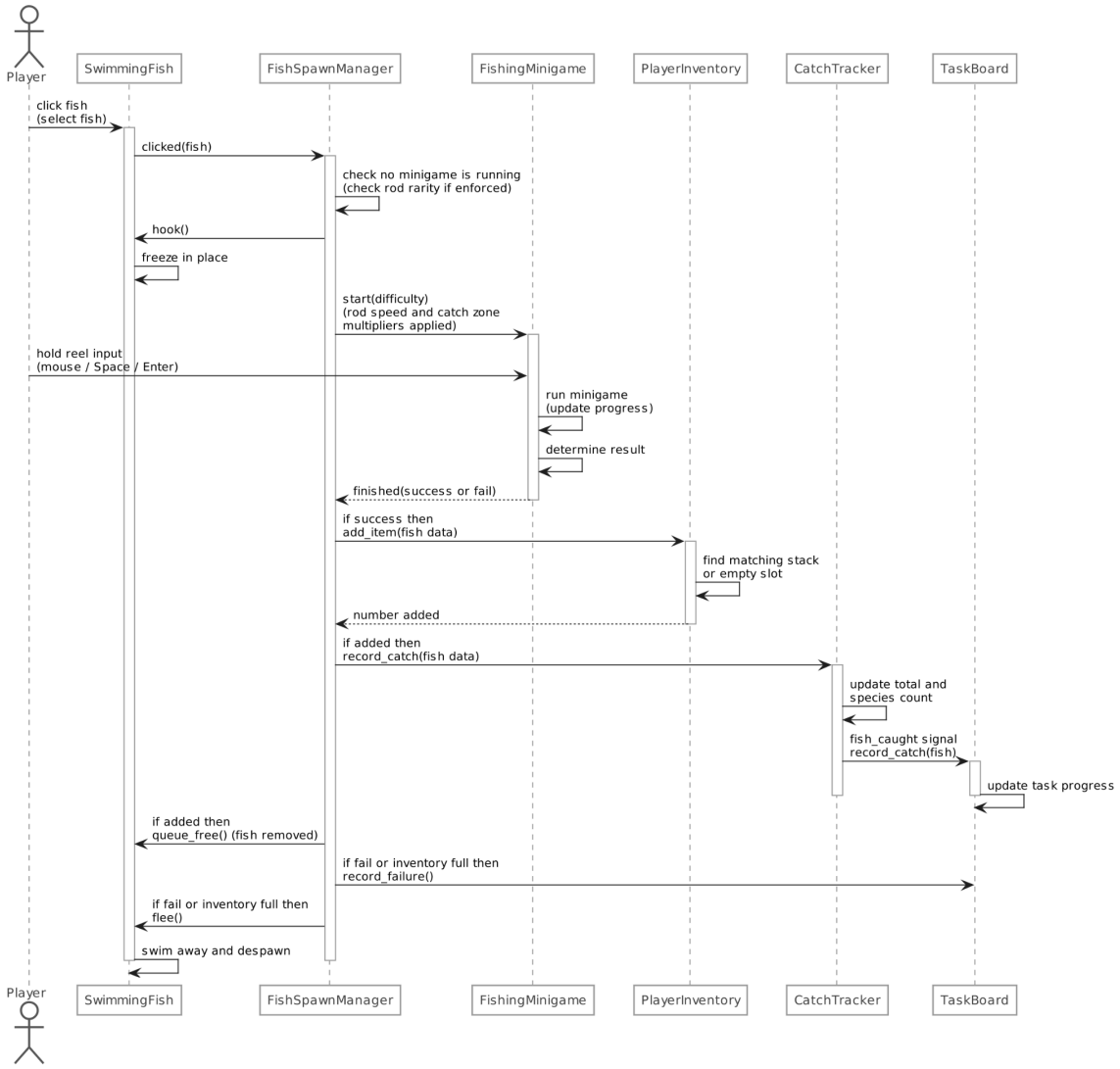
Nonfunctional Requirements: The system keeps the balance and task progress accurate.

5. Class Diagram and/or Sequence Diagrams (15 points)

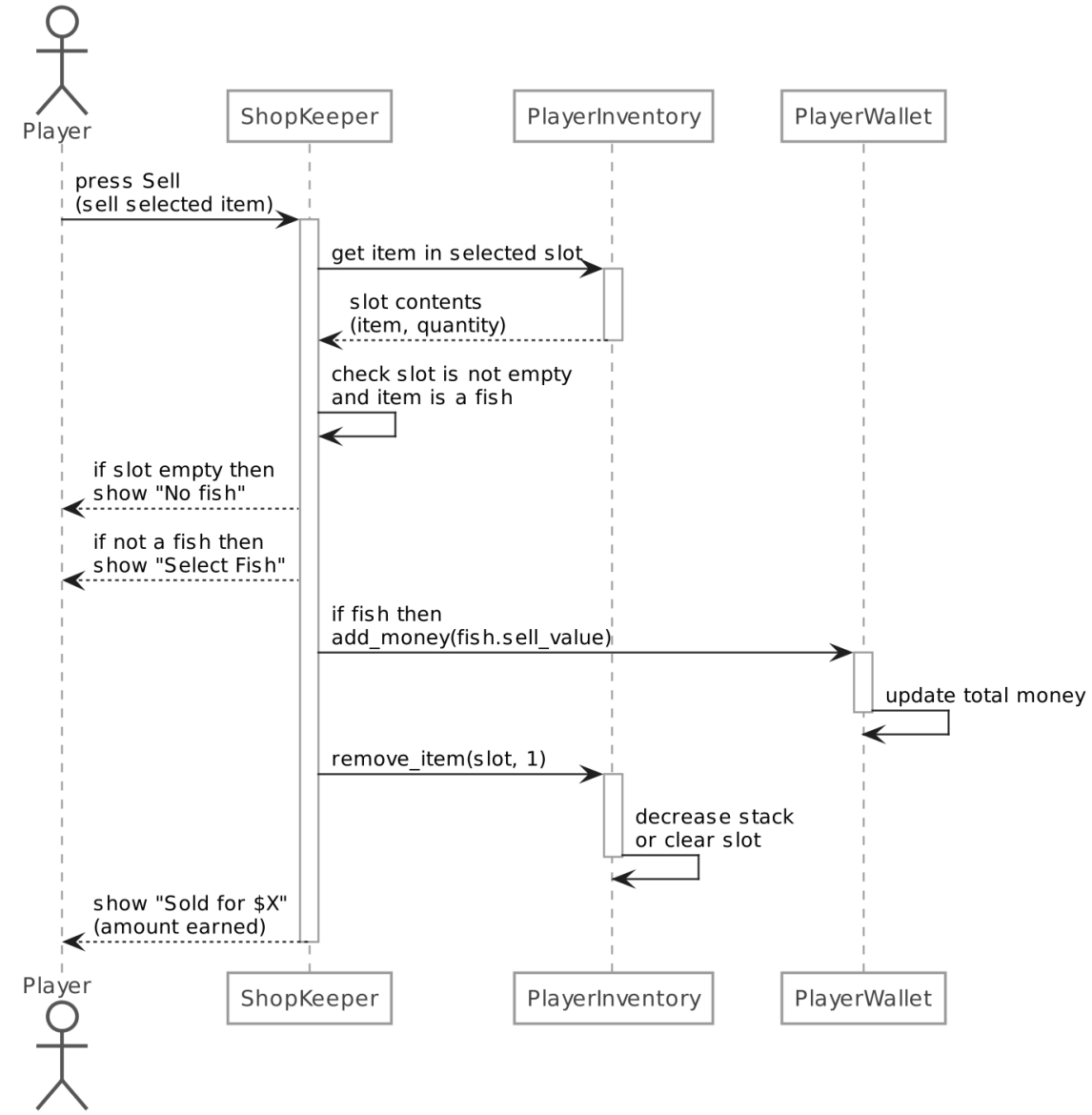
Updated Class diagram



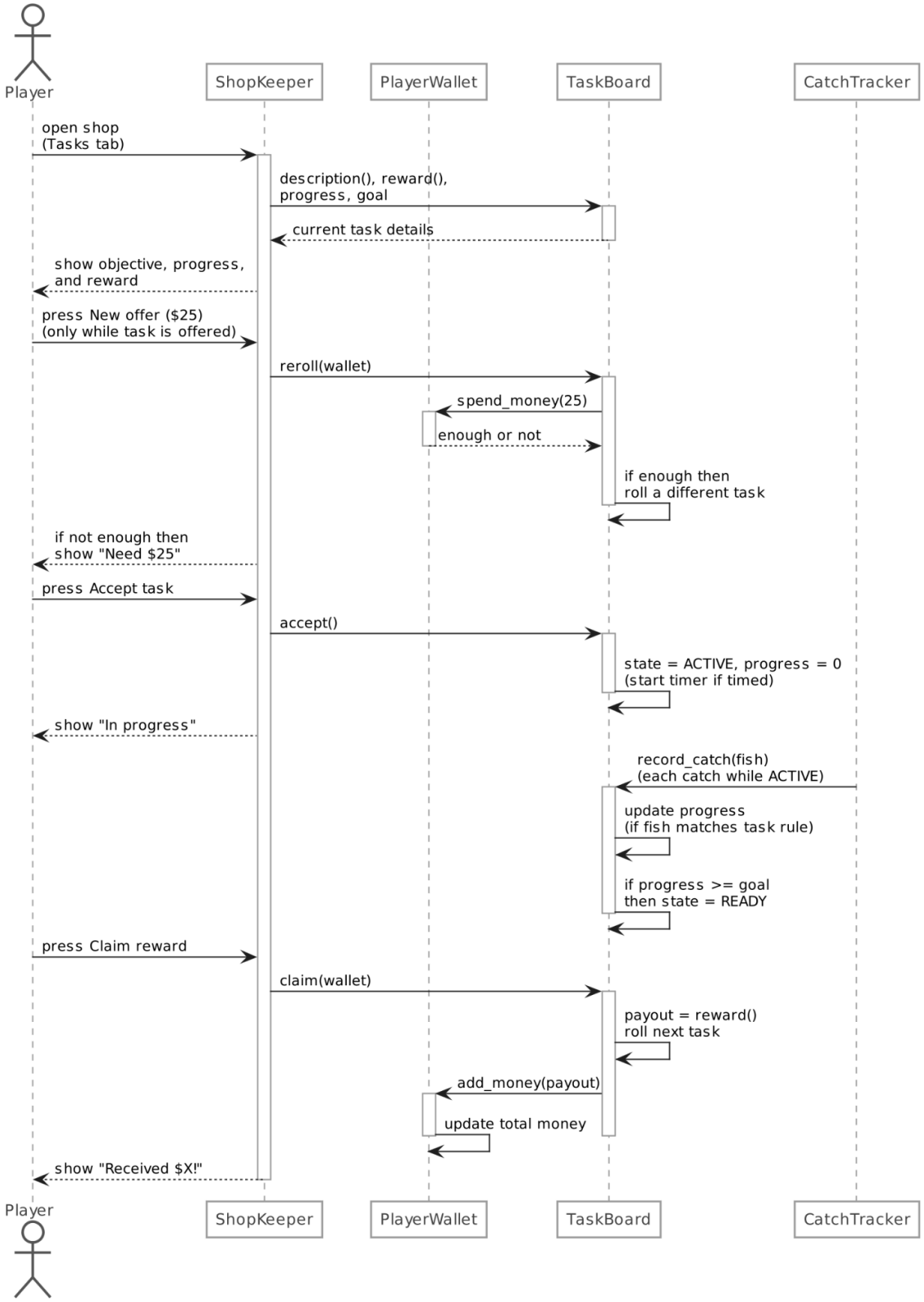
Catch fish sequence diagram



Sell fish sequence diagram



Task board sequence diagram



6. Operating Environment (5 points)

Since Hoo's Fishin' has not yet been released, this section describes the environment it is being developed in and is intended to run on. The game runs on Godot Engine 4.7 using Godot's Compatibility (OpenGL-based) renderer, chosen for its low hardware requirements and support for older or integrated graphics cards. It renders at 384×216 and scales to fill the window, standard for pixel-art games. Our team has developed and tested primarily on Windows, though Godot 4 is cross-platform and also supports macOS and Linux. Hardware requirements are minimal: any computer capable of running the Godot 4 editor with OpenGL 3.3 support, plus a keyboard and mouse (arrow keys/WASD to move, number keys for the hotbar, mouse to interact with fish). The game currently runs through the Godot editor, as no standalone export build exists yet. In development, it also depends on Git/GitHub for version control, though this will not be required by end users. A future increment will include exporting to a standalone executable, after which no additional software will be needed to run the game.

7. Assumptions and Dependencies (5 points)

We are assuming that Godot and the open source tools used in development will remain available and continue to support the versions we are using throughout the project. If these tools changed significantly or became unavailable, it could affect our development process. The project depends on Godot Engine 4.7 as its core third party component. Since Godot is free and open source there are no costs or restrictions to account for. The project also depends on GitHub for version control, and we are assuming its free tier will continue to be sufficient for our needs. Because the game does not rely on internet connectivity or an external database, it is not dependent on any third party servers or APIs, which reduces external risk. We are also assuming that future increments, such as exporting a standalone executable, will not require additional paid tools or licenses. If any of these assumptions are incorrect or change, the requirements described in this document may need to be revisited.